TECHNICAL REFERENCE & CURRICULUM GUIDE

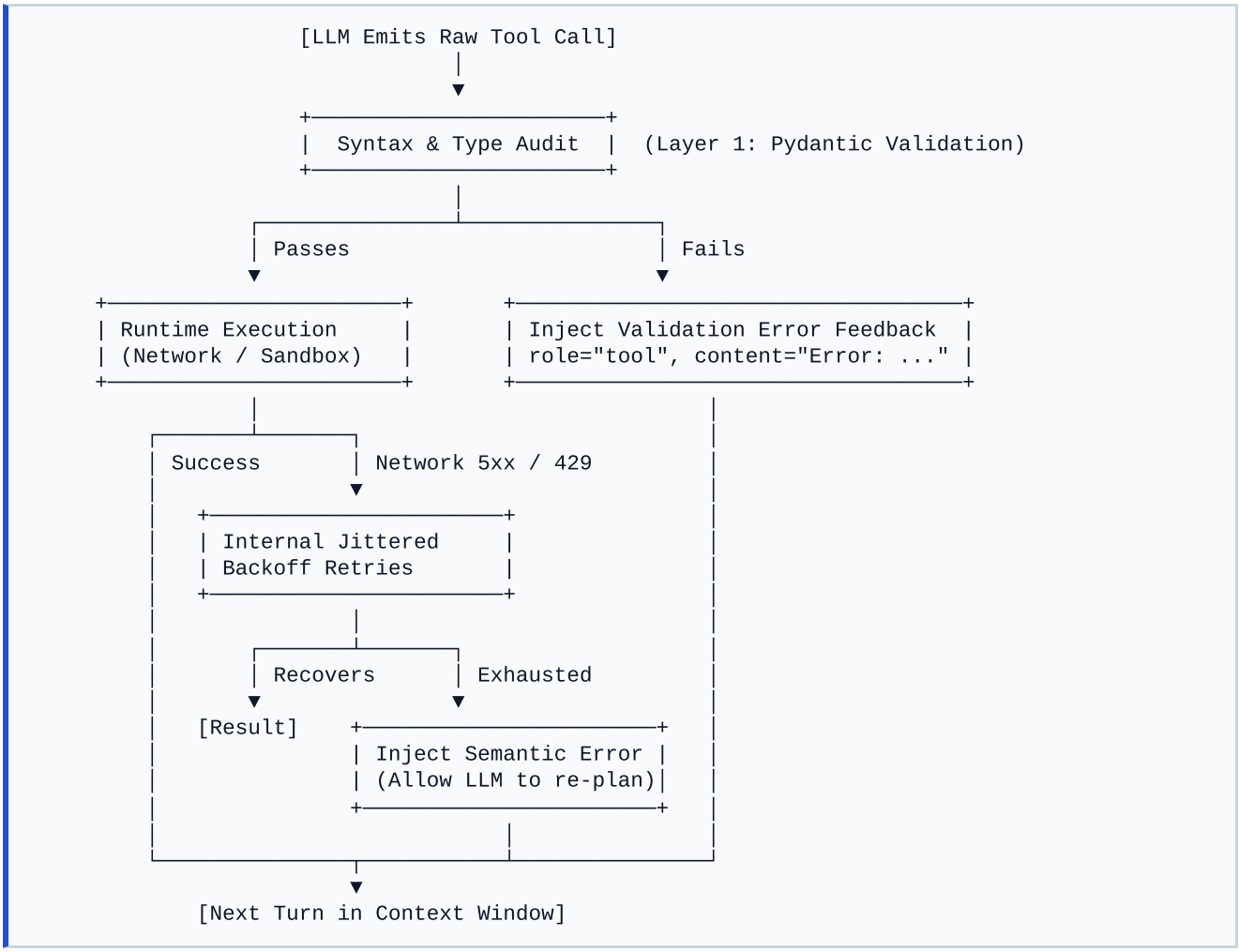
# Module 8 (Part 2): Tool Calling & Agentic Workflows

*Module 8 Part 2 covers the operational, security, economic, and evaluation layers of running tool-calling agents in production environments.*

---

## 8.7 Error Handling, Retries, and Validation Failures in Tool Execution

In production, tool calls fail across multiple layers: the LLM may hallucinate an argument that violates a JSON schema, an external API may return a 503 gateway timeout, or the execution engine may encounter unexpected edge cases. Production agent runtimes treat tool failures as **structured context feedback loops** rather than terminating exceptions.




### 1. The Three Layers of Failure

1. **Schema & Argument Validation Failures (Pre-Execution):** The LLM emits malformed JSON, omits a required parameter, or outputs the wrong data type (e.g., passing a string `"twenty"` instead of an integer `20`).
2. **Transient Infrastructure Failures (Execution Layer):** Downstream REST endpoints throw rate limits (HTTP 429), gateway timeouts (HTTP 504), or dropped database sockets.

3. **Semantic / Domain Failures (Post-Execution):** The syntax is valid and the API responds with HTTP 200, but the business logic rejects the operation (e.g., `{"status": "rejected", "reason": "Insufficient funds in checking account"}`).

## 2. The Self-Correction Feedback Loop

When a tool fails due to malformed parameters or non-transient API rejections, the error should **not crash the Python process**. Instead, the error must be serialized into the message history under `role="tool"` (OpenAI) or `type="tool_result"` (Claude) with descriptive diagnostic details.

The LLM reads its own error output in the next forward pass and uses its reasoning capabilities to self-correct:

- *Bad Pattern:* Returning a generic string like `"Error"`. The model has no context to determine what went wrong and will repeat the identical call.
- *Production Pattern:* Returning actionable, schema-aligned error strings:

```
{
  "status": "validation_error",
  "tool": "transfer_funds",
  "failed_parameter": "currency",
  "received": "bucks",
  "expected": "ISO 4217 code matching ['USD', 'EUR', 'GBP']",
  "guidance": "Please re-read the required ISO code format and re-attempt the call."
}
```

## 3. Production Resilient Tool Execution Engine (Python + Pydantic)

```
import json
import asyncio
from typing import Dict, Any, Callable
from pydantic import BaseModel, ValidationError, Field

class ResilientToolExecutor:
    """
    Executes tool calls with multi-layered validation, deterministic retry
    budgets, and self-correcting context injection.
    """
    def __init__(self, max_transient_retries: int = 3):
        self.registry: Dict[str, Callable] = {}
        self.schemas: Dict[str, type[BaseModel]] = {}
        self.max_transient_retries = max_transient_retries

    def register(self, name: str, schema: type[BaseModel], fn: Callable):
        self.registry[name] = fn
        self.schemas[name] = schema

    async def execute(self, tool_name: str, raw_arguments_str: str) -> str:
        # Layer 1: Parse stringified JSON syntax
        try:
            parsed_args = json.loads(raw_arguments_str)
        except json.JSONDecodeError as e:
            return json.dumps({
                "error_type": "JSON_SYNTAX_ERROR",
                "message": f"Malformed JSON string: {str(e)}. Provide strictly valid JSON."
            })

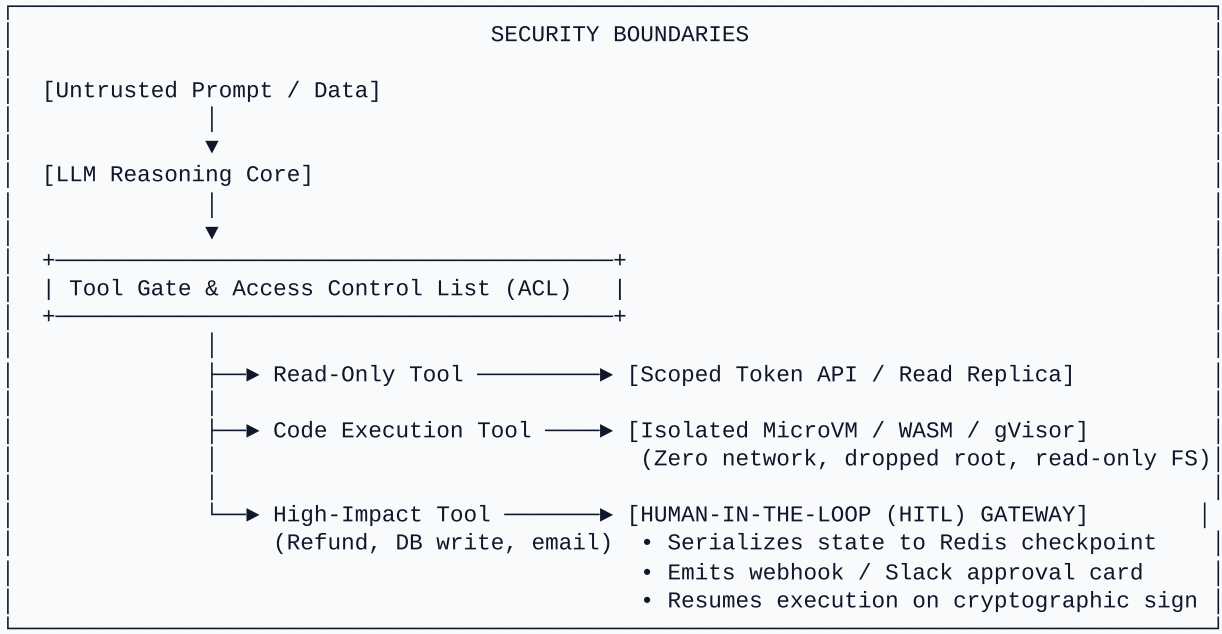
        # Layer 2: Check tool existence
        if tool_name not in self.registry:
            return json.dumps({
                "error_type": "TOOL_NOT_FOUND",
                "message": f"Tool '{tool_name}' does not exist. Available tools:
{list(self.registry.keys())}"
            })

        # Layer 3: Pydantic structural and type validation
        schema_cls = self.schemas[tool_name]
        try:
            validated_model = schema_cls.model_validate(parsed_args)
        except ValidationError as e:
            # Flatten validation errors for clear LLM consumption
            field_errors = [
                {"field": " -> ".join(str(loc) for loc in err["loc"]), "issue": err["msg"]}
                for err in e.errors()
            ]
            return json.dumps({
                "error_type": "PARAMETER_VALIDATION_ERROR",
                "schema_contract": schema_cls.model_json_schema()["properties"],
                "validation_errors": field_errors,
                "remediation": "Correct the parameter types and schema violations specified
above."
            })

        # Layer 4: Execution with automated transient backoff
        fn = self.registry[tool_name]
        for attempt in range(1, self.max_transient_retries + 1):
            try:
                # Support both synchronous and asynchronous callables
                if asyncio.iscoroutinefunction(fn):
                    result = await fn(**validated_model.model_dump())
                else:
```

## 8.8 Security: Sandboxing, Permission Scoping, and Human Approval

Granting an LLM tool access introduces severe security vulnerabilities if untrusted user prompts can trigger arbitrary code execution, exfiltrate data, or mutate production records via prompt injection attacks.




### 1. Code Sandboxing Architectures

When an agent is equipped with a code execution tool (e.g., executing Python for data science or calculating figures), **never use Python's built-in `eval()` or `exec()` directly on the host machine**:

- **Standard Container Pitfall:** Running code inside a standard Docker container running as root allows malicious actors to exploit Linux kernel vulnerabilities or scan the internal container network.
- **MicroVM & Kernel Isolation (Production Standard):**
  - **gVisor (runsc) / Firecracker:** Intercepts system calls in user space, ensuring untrusted Python processes cannot touch the host Linux kernel.
  - **WebAssembly (WASM):** Compiles Python runtimes (e.g., Pyodide) to a WASM sandbox with strictly isolated linear memory and no host file system or network access unless explicitly bridged.
  - **Sandbox Constraints:**
    - Network egress: `None` (air-gapped; disable all socket creation).
    - Execution timeout: Hard ceiling of `5.0 seconds`.
    - Memory limit: Hard ceiling of `256 MB` via cgroups.
    - File system: Ephemeral, mounted in-memory (`tmpfs`), destroyed on process exit.

### 2. Permission Scoping & Principle of Least Privilege

- **Per-User Scoped Tokens:** Tools should never run using a monolithic administrative API key. If User *A* triggers a `search_customer_files` tool, the tool execution context must inherit only User *A*'s OAuth2 JWT or scoped database session.
- **Database Sandboxing:** Expose tools to **Read-Only Replicas** using a dedicated database user with strict `SELECT` permissions on a whitelisted view schema. Never point an agent at production primary databases without row-level security (RLS).

### 3. Human-in-the-Loop (HITL) Approval Gates

Any tool that produces **irreversible side effects** (sending an external email, mutating banking records, issuing refunds, deleting files) must trigger an execution interrupt.

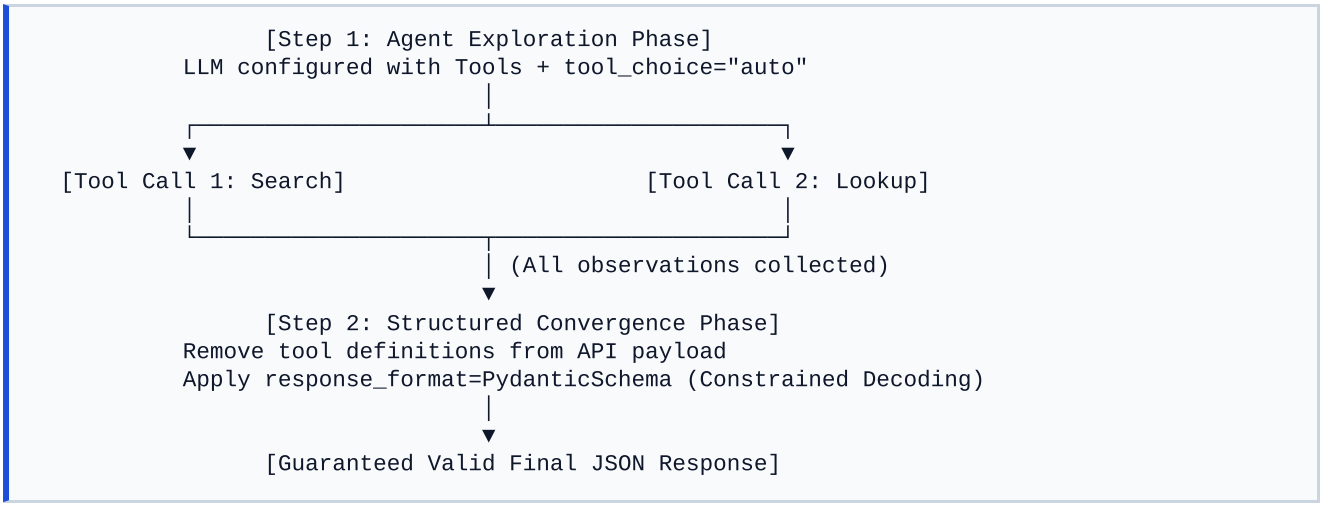
#### The Pause-Resume Serialization Pattern

Because HTTP web servers cannot keep worker threads open waiting for a human manager to click "Approve" in Slack, production agent runtimes use state serialization:

1. **Interrupt Trigger:** When the model calls a tool flagged with `requires_approval = True`, the execution engine halts execution immediately.
2. **State Checkpointing:** The agent serializes its complete state—the conversation history, the pending `tool_call_id`, function name, and proposed arguments—into an external store (e.g., Redis, PostgreSQL).
3. **Approval Notification:** The engine fires a webhook to an operations channel (e.g., a Slack Interactive Message or web dashboard) displaying the exact proposed parameters and a unique `action_id`.
4. **Resolution:**
   - If **Approved:** A webhook endpoint loads the serialized state from the database, runs the tool, appends the result, and triggers the LLM to complete its cycle.
   - If **Rejected:** The webhook endpoint appends a rejection message: `{"status": "declined", "operator_note": "Amount exceeds tier threshold"}`. The agent adjusts its plan accordingly.
   - If **Timed Out:** The gate defaults to a closed state, discarding the pending operation safely.

## 8.9 Combining Structured Outputs with Tool Calling in Production Pipelines

A common production requirement is having an agent explore an environment via dynamic tools, yet guarantee that its final response to the user conforms to a **rigid, validated Pydantic schema** (rather than unstructured markdown text).




### 1. The Mode Conflict Problem

Attempting to force an LLM to simultaneously output a structured JSON schema *and* emit native tool calls in the same forward pass often fails across providers:

- Enforcing a top-level grammar constraint (via Context-Free Grammars/CFGs) restricts the model's logits to match the output schema brackets `{ "answer": ... }`, which directly suppresses the special control tokens needed to trigger tool calls.
- **The Production Solution (Two-Phase Convergence):**
  - **Phase 1 (Agentic Exploration):** Equip the model with its full tool inventory and configure `tool_choice="auto"`. Allow the model to execute multi-step tool loops.
  - **Phase 2 (Structured Synthesis):** Once the model signals it has all necessary data (or when a designated terminal synthesis tool is reached), remove the tools from the payload and make a final call configured with `response_format=FinalReportSchema` using constrained decoding.

## 2. Implementation: Two-Phase Synthesis Pipeline

```
from pydantic import BaseModel, Field
from openai import OpenAI
import json

client = OpenAI()

# Target final schema for end-user ingestion
class MarketIntelligenceReport(BaseModel):
    ticker: str
    current_price: float
    summary: str = Field(description="2-sentence executive summary.")
    sentiment: str = Field(description="BULLISH, BEARISH, or NEUTRAL.")
    risk_factors: list[str] = Field(description="Top 3 extracted risk factors.")

# Mock market data tool
tools = [{
    "type": "function",
    "function": {
        "name": "fetch_financial_metrics",
        "description": "Fetch real-time financial metrics for a company.",
        "parameters": {
            "type": "object",
            "properties": {"ticker": {"type": "string"}},
            "required": ["ticker"]
        }
    }
}]

def run_structured_agent_pipeline(user_prompt: str) -> MarketIntelligenceReport:
    messages = [
        {"role": "system", "content": "You are a financial research agent. Use tools to
gather data."},
        {"role": "user", "content": user_prompt}
    ]

    # --- PHASE 1: Tool Execution Loop ---
    while True:
        response = client.chat.completions.create(
            model="gpt-4o",
            messages=messages,
            tools=tools,
            tool_choice="auto"
        )
        msg = response.choices[0].message
        messages.append(msg)

        if not msg.tool_calls:
            break  # Data gathering complete

        for call in msg.tool_calls:
            # Mock data response
            mock_data = {
                "ticker": "AAPL", "price": 228.50,
                "pe_ratio": 33.2, "risks": ["Supply chain concentration", "Antitrust
litigation", "FX headwinds"]
            }
            messages.append({
                "role": "tool",
                "tool_call_id": call.id,
                "content": json.dumps(mock_data)
            })

    # --- PHASE 2: Structured Convergence Pass ---
```

## 8.10 Cost and Latency Implications of Tool-Heavy Agent Loops

Agentic loops amplify API billing and latency due to their iterative, multi-turn architecture. Understanding these bottlenecks is critical for maintaining scalable economics.

### 1. Cumulative Token Compounding ($O(K^2)$ Scaling)

Every intermediate tool call, parameter dictionary, and stringified observation remains in the context window for all subsequent iterations.

$$\text{Total Billed Tokens} = N_{\text{system}} \cdot K + \sum_{k=1}^{K} \left( \text{Prompt}_k + \sum_{j=1}^{k} \text{Obs}_j \right)$$

If an agent executes 8 tool steps to answer a single question, and each tool returns a 1,500-token JSON payload:

- Turn 1 Prefill: *1,500* tokens.
- Turn 2 Prefill: *3,000* tokens.
- Turn 8 Prefill: *12,000* tokens.
- **Cumulative Prefill Volume:** Over ***54,000* tokens** are billed to upstream inference providers for a single interaction, compounding latency at each prefill forward pass.

### 2. Network Latency Compounding

Wall-clock time for an agent run with $K$ sequential tool calls is lower-bounded by:

$$\text{Total Latency} = \sum_{k=1}^{K} \left( \text{TTFT}_{\text{prefill},\,k} + \text{TimePerToken} \times M_k + \text{NetworkLatency}_{\text{tool},\,k} \right)$$

If each LLM generation step takes 1.2 seconds and each external tool call requires 400ms of network I/O, a 5-step sequential agent run takes ***8.0* seconds minimum**, making it unsuitable for synchronous web UX without progress streaming.

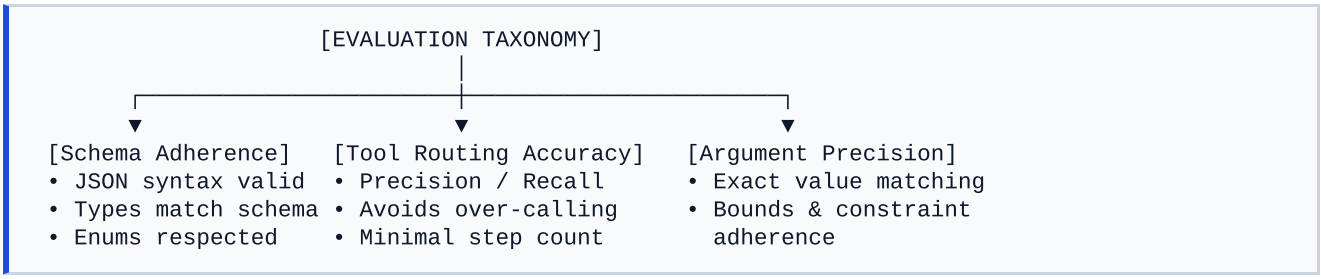
### 3. Production FinOps & Optimization Guidelines

1. **Tool Output Minification & Filtering:** Never return raw database rows or unparsed HTML into `role="tool"`. Strip unused metadata fields, unescaped whitespace, and redundant keys in Python *before* serializing to JSON.
2. **Aggressive Context Compaction / Truncation:** For tools returning long search logs, retain only the top 3 relevant snippets or generate a condensed summary using a cheap Small Language Model (SLM) before injecting into the main agent's working context.

3. **Prefix Caching Alignment:** Keep tool schema definitions and system prompts byte-identical across calls. Do not dynamically add or remove tools halfway through a multi-step loop; modifying the tool array invalidates the upstream KV-cache prefix hash and destroys prompt caching discounts.
4. **Tool Pruning & Router Gating:** Supplying 40 tools in a single prompt degrades tool-selection accuracy and consumes 3,000+ tokens per request just for schemas. Use a **Semantic Tool Router**: run a fast embedding lookup or a lightweight SLM classifier to dynamically bind only the 3–5 tools relevant to the user's specific request.

## 8.11 Testing and Evaluating Tool-Calling Reliability

Evaluating an agent that interacts with tools requires testing intermediate reasoning steps and tool execution traces, rather than evaluating only the final natural language output.




### 1. Key Evaluation Metrics

- **Schema Adherence Rate (%):** Measures how often the model emits tool arguments that parse into the target schema without throwing Pydantic or JSON validation errors. Frontier models achieve *>99%*; smaller open models often require grammar-constrained decoding.
- **Tool Selection Accuracy (Precision & Recall):**
  - **Precision:** Did the agent avoid calling tools when the prompt could be answered directly from parametric memory?
  - **Recall:** Did the agent call the appropriate tool when required external context was requested?
- **Argument Correctness / Extraction Error Rate:** Checks whether extracted argument values match ground truth (e.g., extracting `"2026-11-05"` vs. `"tomorrow"`).
- **Step Efficiency (Trajectory Optimization):** Evaluates whether the agent reached the solution in the minimal necessary steps, or wandered through redundant loops.

### 2. Automated Evaluation Suite (pytest + Unit Testing Pattern)

```
import pytest
import json
from pydantic import BaseModel

# Ground-truth evaluation dataset
BENCHMARK_CASES = [
    {
        "query": "What is the balance of account 9912?",
        "expected_tool": "get_account_balance",
        "expected_args": {"account_id": 9912}
    },
    {
        "query": "Cancel subscription sub_441 immediately.",
        "expected_tool": "cancel_subscription",
        "expected_args": {"subscription_id": "sub_441", "immediate": True}
    }
]

def test_tool_calling_accuracy(mock_llm_client):
    for case in BENCHMARK_CASES:
        # Run test inference
        response = mock_llm_client.get_tool_call(case["query"])

        # 1. Assert tool was emitted
        assert response.tool_calls is not None, f"Failed to call any tool for:
{case['query']}"
        assert len(response.tool_calls) == 1

        call = response.tool_calls[0]

        # 2. Assert correct tool selection (Recall)
        assert call.function.name == case["expected_tool"], (
            f"Incorrect tool selected. Expected {case['expected_tool']}, got
{call.function.name}"
        )

        # 3. Assert argument extraction accuracy
        actual_args = json.loads(call.function.arguments)
        for key, expected_val in case["expected_args"].items():
            assert key in actual_args, f"Missing required argument: {key}"
            assert actual_args[key] == expected_val, (
                f"Argument mismatch for {key}. Expected {expected_val}, got
{actual_args[key]}"
            )
```

## 8.12 Weekly Project: Production-Grade Tool-Calling Assistant

Below is a complete, runnable, production-grade agent implementation featuring:

- **5+ Live Real-World Tools:**
  1. `calculator` (Secure math expression evaluation using Python Abstract Syntax Trees).
  2. `system_clock` (Time-zone aware UTC and local ISO-8601 timestamps).
  3. `database_query` (Mock relational lookup for customer records).
  4. `http_fetcher` (Sanitized outbound HTTP GET requests with domain whitelisting).
  5. `request_human_approval` (Human-in-the-loop gate for dangerous mutating operations).

- **Defensive Architecture:** Complete schema enforcement, Pydantic parsing, multi-step dependency orchestration, error capture, and execution ceilings.

```
import ast
import json
import operator
from datetime import datetime, timezone
from typing import Dict, Any, List, Literal
from pydantic import BaseModel, Field, ValidationError
from openai import OpenAI

# ================================================================================
# 1. SECURE TOOL IMPLEMENTATIONS & PYDANTIC CONTRACTS
# ================================================================================

# Tool 1: AST-Based Math Evaluator (Zero eval() usage)
class CalculatorInput(BaseModel):
    expression: str = Field(description="Math expression string, e.g. '(50 * 12) + 140'.
Supports +, -, *, /, %.")

def safe_math_eval(expression: str) -> str:
    """Evaluates arithmetic expressions safely using AST parsing."""
    allowed_operators = {
        ast.Add: operator.add, ast.Sub: operator.sub,
        ast.Mult: operator.mul, ast.Div: operator.truediv,
        ast.Mod: operator.mod, ast.USub: operator.neg
    }
    def eval_node(node):
        if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
            return node.value
        elif isinstance(node, ast.BinOp) and type(node.op) in allowed_operators:
            return allowed_operators[type(node.op)](eval_node(node.left),
eval_node(node.right))
        elif isinstance(node, ast.UnaryOp) and type(node.op) in allowed_operators:
            return allowed_operators[type(node.op)](eval_node(node.operand))
        raise ValueError("Unsupported or insecure mathematical expression.")

    try:
        parsed_ast = ast.parse(expression, mode='eval')
        result = eval_node(parsed_ast.body)
        return json.dumps({"expression": expression, "result": result})
    except Exception as e:
        return json.dumps({"error": f"Evaluation error: {str(e)}"})

# Tool 2: System Clock
class ClockInput(BaseModel):
    format: Literal["iso", "human"] = Field(default="iso", description="Timestamp
representation format.")

def get_system_clock(format: str = "iso") -> str:
    """Returns current UTC and local timestamps."""
    now = datetime.now(timezone.utc)
    if format == "human":
        return json.dumps({"current_time_utc": now.strftime("%A, %B %d, %Y %H:%M:%S UTC")})
    return json.dumps({"current_time_iso": now.isoformat()})

# Tool 3: Scoped Database Query
class DBQueryInput(BaseModel):
    user_id: int = Field(description="Customer integer identifier.")
    entity: Literal["profile", "billing", "usage"] = Field(description="Entity slice to
inspect.")

def mock_database_query(user_id: int, entity: str) -> str:
    """Queries customer database slices."""
    mock_db = {
        1001: {
            "profile": {"name": "Alice Vance", "tier": "Enterprise", "email":
"alice@vance.io"},
            "billing": {"balance_due_usd": 150.00, "status": "Current"}
```

## Technical Summary of Module 8 Part 2

| Topic | Core Engineering Focus | Primary Vulnerability or Failure | Production Defense Architecture |
|---|---|---|---|
| **8.7 Error Handling** | Pre-execution validation & runtime retries | Silent crashes, unhandled 5xx errors, infinite error loops | Pydantic schema validation, exponential backoff, structured self-correction injection |
| **8.8 Security & HITL** | Sandboxing & approval gates | Arbitrary code execution, unauthorized mutations via injection | gVisor/WASM sandboxes, pause-resume state serialization, human approval gates |
| **8.9 Structured Output** | Combining tools with guaranteed JSON schemas | Constrained decoding suppressing tool-call tokens | Two-phase pipeline: unconstrained tool loop followed by grammar-constrained synthesis |
| **8.10 FinOps & Latency** | Managing agent execution costs and response times | $O(K^2)$ context compounding, latency multiplication | Minified tool payloads, semantic tool routing, invariant system prefixes |
| **8.11 Testing & Evals** | Automated agent reliability verification | Flaky benchmarks, undetected argument drift | Pytest unit assertions, trace-based span evaluation, trajectory efficiency metrics |
| **8.12 Weekly Project** | Production tool-calling architecture | Single-point failure in tool infrastructure | AST parsing, scoped database abstractions, and unified state management |